SplitTogether Fullstack Guide (Beginner-Friendly)

1) What this app is

SplitTogether is a web app where a frontend UI talks to backend API routes and a PostgreSQL database.
Users can sign up, log in, create rooms, add expenses, and track balances.

2) Stack used and why

- Next.js 14 (App Router): one project for UI pages and backend API routes, simpler for MVP speed.
- TypeScript: catches mistakes earlier and makes refactors safer.
- React + Tailwind CSS: reusable UI components and fast styling.
- PostgreSQL: reliable relational database for connected data (users, groups, expenses).
- Prisma ORM: type-safe DB queries and easy schema management.
- NextAuth (credentials): session/login handling integrated with Next.js.
- Zod validation: clear request validation and safer API inputs.
- Docker Compose: predictable local Postgres setup.

3) How frontend and backend are linked in this app

Flow: user action in UI -> fetch() call -> /api route -> validation/auth checks -> Prisma DB query -> JSON response -> UI updates.
Example (create room):

- UI component submits room name from dashboard form.
- API route POST /api/groups validates input, checks session, creates Group + GroupMember + ActivityLog.
- UI gets returned room id and navigates to /groups/{id}.

4) Key folders to know

- web/app/: pages and API routes.
- web/components/: reusable UI pieces.
- web/lib/: shared logic (session helpers, money helpers, balance math).
- web/prisma/schema.prisma: data model.
- web/prisma/seed.ts: demo seed data.
- web/middleware.ts: route protection.

5) Authentication in simple terms

- Signup endpoint stores password hash (not plain text).
- Login checks password hash and creates session token.
- Protected pages (dashboard/groups) redirect to login if not authenticated.
- API routes also check session server-side before writing data.

6) Data model basics

Core tables: User, Group, GroupMember, Expense, ExpenseShare, Settlement, ActivityLog.
Important rule: money is stored as integer cents to avoid decimal rounding bugs.

7) Running the app (one command)
From repo root: zsh start-web.zsh
What it does: starts Postgres, ensures web/.env exists, installs deps, runs Prisma generate + db push, starts Next dev server.
Useful flags: --reset-db (wipe + reseed), --seed, --skip-install, --port 3001.

8) How to build new features yourself (repeatable checklist)
1. Write the feature behavior in plain English first.
2. Update schema.prisma only if data model must change.
3. Add input rules in web/lib/validation.ts.
4. Implement/extend API route in web/app/api/...
5. Enforce auth + room membership checks.
6. Build/adjust UI component and connect to API with fetch.
7. Handle success and error messages clearly for users.
8. Run validation: npm run lint && npm run build.
9. Smoke test: signup/login/create room/protected route/invalid input.

9) Alternatives and tradeoffs
- Separate frontend and backend services: more scalable boundaries, more complexity.
- Firebase/Supabase-first: faster setup, less control for complex business rules.
- Keep Django backend + Next frontend: mature backend features, but two active stacks to maintain.
- Rails/Laravel monolith: productive fullstack experience, but larger ecosystem shift.

10) Security and production notes
- NEXTAUTH_SECRET must be set in production.
- Validate all incoming API payloads (already done with Zod).
- Never trust client-side checks alone; always enforce server-side authorization.
- Keep dependencies updated; audit findings should be tracked and patched in planned upgrades.

11) Learning path for a beginner
Week 1: HTTP basics (request/response/status code), JSON, DB table relationships.
Week 2: React basics (components/state/forms) and reading existing UI components.
Week 3: API route basics + Prisma queries + auth checks.
Week 4: deliver one small full feature end-to-end (example: room description).
Practice rule: always explain the feature in plain English before coding.

12) Quick glossary
Frontend: what users see.
Backend: server logic and rules.
API: doorway between frontend and backend.
ORM: library that translates code to DB queries.
Session: logged-in state.
Migration/schema change: database structure update.

Seed: demo data inserted into DB.

End of guide.